A Project Report on

# Coding Projects Folder Analyzer

Submitted in partial fulfilment of the requirements of the degree of

BACHELOR OF ENGINEERING

By


Prathamesh Kolhe: 23106132

Bhavika Kadam: 23106079

Soham Jathar: 23106084

Onkar Deshmukh: 23106006


Under the Guidance of

Mr. Vijesh Mundokalam


**Department of Computer Science & Engineering**
**(Artificial Intelligence & Machine Learning)**

**A. P. SHAH INSTITUTE OF TECHNOLOGY, THANE**


(2025-2026)

**A. P. SHAH INSTITUTE OF TECHNOLOGY, THANE**

# CERTIFICATE

This is to certify that the project entitled **"Coding Projects Folder Analyzer"** is a bonafide work of **Prathamesh Kolhe (23106132), Bhavika Kadam (23106079), Soham Jathar (23106084), Onkar Deshmukh (23106006)** submitted to the University of Mumbai in partial fulfilment of the requirement for the award of the degree of **Bachelor of Engineering** in **Computer Science & Engineering (Artificial Intelligence & Machine Learning)**

Mr. Vijesh Mundokalam
Project Guide

Prof. Yogeshwari Hardas
Project Co-Ordinator

Dr. Jaya Gupta
Head of Department

Dr. Uttam D Kolekar
Principal

**A. P. SHAH INSTITUTE OF TECHNOLOGY, THANE**

# Project Report Approval for T. E.

This project report entitled ***Coding Projects Folder Analyzer*** by ***Prathamesh Kolhe, Bhavika Kadam, Soham Jathar, Onkar Deshmukh*** is approved for the degree of ***Bachelor of Engineering*** in **Computer Science & Engineering (Artificial Intelligence & Machine Learning)**, ***2025-26***.

| Examiner Name | Signature |
|---|---|
| 1. _____ | _____ |
| 2. _____ | _____ |

Date:

Place:

# Declaration

We declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Prathamesh Kolhe
23106132

Bhavika Kadam
23106079

Soham Jathar
23106084

Onkar Deshmukh
23106006

Date:

# Abstract

Coding project analysis plays a critical role in understanding software systems and improving developer productivity. However, conventional code exploration tools are often limited to basic file navigation, lack structural insights, and require significant manual effort, limiting their effectiveness for developers working with unfamiliar codebases. This paper presents a Coding Project Folder Analyzer that integrates structured graph-based analysis with an intuitive visualization interface. The proposed system models project structures as graphs, where nodes represent files or modules and edges define dependency relationships to ensure clear understanding of code flow. A metadata-driven analysis framework supports dynamic operations such as dependency detection, file ranking, framework identification, execution flow tracing, and structural mapping. Project data is processed and represented in structured formats, enabling persistence, portability, and reuse of analysis results. The execution engine employs graph traversal techniques to ensure accurate dependency mapping and correct sequential interpretation of code relationships. The system supports multi-language project analysis and visualization using interactive graph representations. Experimental validation using sample coding projects demonstrates reliable analysis accuracy and improved performance in understanding code structures. The proposed approach bridges the gap between traditional code navigation tools and intelligent developer assistance platforms, offering a lightweight and extensible solution for educational and small-scale software engineering applications.

**Keywords:** Code Analysis; Dependency Graph; AST Parsing; PageRank; Software Visualization; Static Analysis; Developer Productivity; Graph-Based Modelling

# CONTENTS

1. Introduction.....1
   1.1 Background.....1
   1.2 Motivation.....3
   1.3 Aim & Scope of the Project.....4
   1.4 Relevance to Sustainable Development Goals.....5
2. Literature Survey.....6
   2.1 Foundations of Code Analysis Systems.....7
   2.2 Code Dependency Analysis and Graph Processing.....8
   2.3 Node-Based and Graph-Based Visualization.....8
   2.4 No-Code and Low-Code Development Platforms.....9
   2.5 Static Code Analysis Techniques.....9
   2.6 Data Visualization and Visual Analytics.....9
3. Limitations in Existing Systems.....11
4. Problem Statement & Objectives.....13
   4.1 Problem Statement.....13
   4.2 Objectives.....14
5. Proposed System & Methodology.....15
   5.1 System Overview.....15
   5.2 System Architecture.....17
   5.3 Workflow Modeling Methodology.....19
6. Experimental Setup.....24
   6.1 Frontend Setup.....24
   6.2 Backend Setup.....25
   6.3 Development Environment:.....26
7. Results & Discussion.....28
   7.1 Results.....28
   7.2 Demonstration.....32
   7.3 Discussion.....33
   7.4 Limitations.....34
8. Conclusion & Future Scope.....35
9. References.....37

# LIST OF FIGURES

Figure 1: Overview of the system flow.....16
Figure 2: Overview of the System Architecture..... 17
Figure 3: Project Analysis Dashboard & Overview..... 29
Figure 4: AI-Powered Intelligent Summary.....30
Figure 5: Dependency Graph.....31
Figure 6: Project Summary & Key Modules.....32

# ABBREVIATIONS

| | |
|---|---|
| AI | Artificial Intelligence |
| API | Application Programming Interface |
| AST | Abstract Syntax Tree |
| BFS | Breadth-First Search |
| CORS | Cross-Origin Resource Sharing |
| D3 | Data-Driven Documents |
| DAG | Directed Acyclic Graph |
| DOM | Document Object Model |
| HTTP | Hypertext Transfer Protocol |
| IDE | Integrated Development Environment |
| JSON | JavaScript Object Notation |
| LOC | Lines of Code |
| LLM | Large Language Model |
| ML | Machine Learning |
| NLP | Natural Language Processing |
| OS | Operating System |
| REST | Representational State Transfer |
| SDG | Sustainable Development Goal |
| UI | User Interface |
| UUID | Universally Unique Identifier |
| Vite | Frontend Build Tool (French for "Fast") |

# Chapter 1

# Introduction

## 1.1 Background

In modern software development environments, organizations and developers work with large and complex codebases originating from various sources such as enterprise applications, open-source repositories, web platforms, and cloud-based systems. Before such codebases can be effectively understood, maintained, or extended, developers must analyse their structure, dependencies, and functionality through several steps including code exploration, dependency identification, structural mapping, and documentation. These processes ensure that unstructured and unfamiliar source code is transformed into a structured and interpretable format suitable for comprehension, debugging, and further development activities.

Coding project analysis systems play a fundamental role in assisting developers with understanding software architecture and code relationships. These systems analyse project folders, extract structural and functional information, and represent dependencies between files and modules to improve clarity and navigation. Over the years, code analysis technologies have evolved significantly from simple file browsing tools to advanced graph-based systems capable of handling complex code relationships and large-scale software structures.

Code analysis workflows typically consist of multiple interdependent operations such as dependency extraction, structural mapping, code parsing, feature detection, and relationship analysis. These workflows are commonly represented using graph-based models where individual files or modules are treated as nodes and their dependencies are represented as edges [1]. Such representations allow code analysis systems to manage structural relationships, maintain dependency mappings, and ensure accurate interpretation of code flow within the project.

Despite their importance, traditional code exploration tools often involve manual navigation processes and require significant technical expertise to understand complex codebases. Additionally, many existing approaches rely heavily on reading source code line by line or using limited documentation, which can introduce cognitive overhead and increase the time required for understanding large projects. These complexities can create barriers for developers who lack prior familiarity with the codebase but still need to analyse and work with it effectively.

Recent advancements in human-centred computing and visual analytics have demonstrated that graphical interfaces can significantly improve the usability of complex software systems. Visual representations enable users to better understand relationships, dependencies, and execution flows within code structures. As highlighted in visual analytics research [3], interactive visualization combined with analytical reasoning allows users to interpret and explore complex systems more intuitively.

Furthermore, the growing adoption of low-code and no-code development platforms has shown that visual programming environments can reduce technical barriers and enable a broader range of users to participate in software development tasks [5]. These platforms use graphical interfaces, reusable components, and visual modelling techniques to simplify application understanding while maintaining functional capabilities.

Motivated by these developments, there is an increasing need for intelligent code analysis systems that combine structured analysis mechanisms with intuitive visual interfaces. Integrating graph-based modelling with interactive visualization can make codebase understanding more accessible while preserving the accuracy, dependency management, and structural clarity required in modern software engineering environments.

## 1.2 Motivation

The rapid growth of software development across industries has significantly increased the demand for efficient code analysis and comprehension mechanisms. Developers today work with large and complex codebases generated from multiple sources such as enterprise applications, open-source repositories, web platforms, and cloud-based systems. Before such codebases can be effectively understood for maintenance, debugging, or feature development, they must undergo a sequence of structured analysis steps including code parsing, dependency extraction, structural mapping, and feature identification. Code analysis systems therefore play a critical role in preparing unfamiliar codebases for efficient understanding and ensuring consistency in software comprehension across development teams.

Despite their importance, conventional code exploration tools are often designed with limited functionality and require significant manual effort to analyse and understand complex code structures. Many existing approaches rely heavily on reading source code, navigating file hierarchies, or using basic IDE features that demand strong programming knowledge from developers. As discussed in software engineering research [1], understanding and analysing codebases involves complex conceptual mapping, dependency tracking, and structural interpretation. This complexity can make codebase analysis difficult for users who do not possess extensive experience with the specific project or programming environment.

Furthermore, traditional approaches to code understanding frequently lack automated mechanisms for representing relationships between files and modules, which can increase cognitive load and time required for analysis. While research has proposed various techniques for improving code analysis efficiency, these approaches primarily focus on backend processing or algorithmic optimization rather than improving accessibility and usability for developers [2]. As a result, there remains a gap between powerful code analysis techniques and user-friendly systems that allow intuitive understanding of software structures.

At the same time, advancements in visual analytics and human-computer interaction have demonstrated the advantages of graphical interfaces for representing complex systems. Visual representations allow users to understand relationships, dependencies, and execution flows more easily compared to traditional text-based approaches. According to visual analytics research [3], interactive graphical representations enhance cognitive understanding by exposing structural relationships in an intuitive and interpretable manner.

Additionally, the emergence of low-code and no-code development platforms has highlighted the potential of visual programming environments to simplify software development and analysis processes. These platforms enable users to interact with systems through graphical components and structured representations rather than manual coding, thereby lowering the technical barrier for understanding complex software projects [5]. However, many existing tools either oversimplify code analysis or limit customization capabilities, which can restrict their effectiveness for detailed dependency analysis and structural interpretation tasks.

These challenges motivate the development of an intelligent code analysis system that combines the reliability of structured analysis techniques with the usability of visual interfaces. A graph-based Coding Project Folder Analyzer can provide users with an intuitive environment for exploring and understanding codebases while maintaining clear dependency mapping and accurate structural representation. By integrating graph-based modelling with interactive visualization, the proposed approach aims to bridge the gap between technically complex code analysis methods and accessible developer tools.

## 1.3 Aim & Scope of the Project

The primary aim of this project is to design and develop a Node-Based Visual ETL Workflow Creator that simplifies the process of constructing and executing data transformation pipelines. The system aims to provide an intuitive graphical interface that allows users to visually create ETL workflows without requiring extensive programming knowledge, while still maintaining structured execution and dependency management.

The scope of the project includes the development of a web-based platform that enables users to upload and analyse project folders using graph-based techniques. The system supports multiple programming environments and performs operations such as dependency extraction, file importance ranking, execution flow tracing, and framework detection. Additionally, the system provides visualization features that allow users to explore relationships between files and modules, along with automated summaries to enhance understanding.

The project primarily focuses on static analysis of small to medium-sized codebases, making it suitable for educational purposes, developer onboarding, research experimentation, and software maintenance tasks.

## 1.4 Relevance to Sustainable Development Goals

The proposed Coding Project Folder Analyzer contributes to the advancement of digital innovation and knowledge accessibility, aligning with several United Nations Sustainable Development Goals (SDGs). In particular, the project supports SDG 9: Industry, Innovation and Infrastructure by promoting the development of intelligent software analysis tools that improve the efficiency and accessibility of code understanding and software engineering workflows.

Additionally, the system contributes to SDG 4: Quality Education by providing an accessible platform for learning and experimenting with code analysis, software structure, and dependency visualization. The visual and interactive approach simplifies complex software engineering concepts, enabling students, developers, and researchers

By lowering technical barriers and encouraging broader participation in software development and analysis, the proposed system supports innovation, learning, and technological development within modern digital environments.

# Chapter 2

# Literature Survey

The rapid growth of software development systems has led to significant research in the fields of code analysis, dependency modelling, software visualization, and intelligent developer assistance platforms. Numerous studies have explored different aspects of software analysis technologies, ranging from foundational code comprehension techniques to advanced graph-based modelling approaches and modern low-code development paradigms. Understanding these developments is important for identifying existing solutions, technological advancements, and research gaps that motivate the development of improved code analysis systems. To systematically analyse prior research relevant to the proposed Coding Project Folder Analyzer, the literature reviewed in this study has been organized into six thematic categories. Each category focuses on a specific aspect of code analysis and related technologies that contribute to the design and implementation of the proposed system.

The first category, Foundations of Code Analysis Systems, discusses the fundamental concepts of code comprehension, their role in software engineering, and the evolution of code analysis technologies. The second category, Code Dependency Analysis and Directed Graph Processing, examines graph-based modelling approaches and the use of graph structures for managing relationships and dependencies within software projects.

The third category, Node-Based Visual Programming, explores visual programming paradigms where computational workflows are represented through interconnected graphical nodes rather than textual code. The fourth category, No-Code and Low-Code Data Processing Platforms, reviews platforms that aim to simplify software and data pipeline development through graphical interfaces and reusable components.

The fifth category, Static and Dynamic Code Analysis Techniques, focuses on research related to analysing program behaviour and structure, highlighting the challenges and architectural considerations associated with large-scale software systems. Finally, the sixth category, Data Visualization and Visual Analytics, examines techniques that integrate visual representations with analytical reasoning to improve user interaction and understanding of complex code structures.

By organizing the literature into these six sections, the review provides a structured understanding of the existing research landscape while identifying the technological gap between complex code analysis techniques and accessible developer tools, which motivates the development of the proposed Coding Project Folder Analyzer.

## 2.1 Foundations of Code Analysis Systems

Code analysis systems are fundamental components of modern software engineering practices [7, 10]. The code analysis process integrates information from multiple source files by extracting structural and functional details, analysing relationships, and representing them in a structured format for better understanding. Code analysis technologies have evolved significantly over time, with various architectural and modelling advancements proposed in literature [11, 13].

Previous research studies [1] provide insights into workflow and system modelling concepts, emphasizing structured representation, logical organization, and performance considerations in complex systems. They describe such systems as directed graphs of interrelated components and highlight research challenges in correctness, scalability, and optimization. Simitsis et al.

[2] present a broader perspective on system evolution, outlining the transition from simple manual analysis techniques to advanced automated and graph-based approaches. Their study identifies key challenges such as dependency mapping, structural interpretation, and efficient analysis of large-scale systems.

Software analysis research [6] further discusses inefficiencies in traditional approaches, particularly the reliance on manual inspection, redundant processing, and increased cognitive overhead. Their work introduces alternative approaches that leverage structured representations and optimization techniques to improve analysis efficiency. Despite these advancements, most existing systems.

## 2.2 Code Analysis Systems and Graph-Based Processing

Conceptual modeling methodologies have been introduced to structure ETL process design [12]. ETL workflows can be formally represented using logical graph-based models [8, 14]. Modern ETL engines model workflows as Directed Acyclic Graphs (DAGs), where nodes represent transformation tasks and edges define execution dependencies. Ali and Wrembel [1] describe architecture graphs and activity graphs used to represent ETL processes formally. Such graph-based models enable optimization strategies including task reordering, parallelization, and cost-based execution tuning. Cost-based optimization and transformation reordering techniques have been proposed to improve ETL performance [9]. Simitsis et al. [2] highlight optimization approaches that treat ETL design as a state-space search problem, enabling performance improvement through transformation equivalence rules. Guo et al. [6] extend this concept through their CCEVP (Cloud Computing Based Effective–Virtual–Physical) layered architecture, which integrates physical, virtual, and effective layers to manage heterogeneous data integration efficiently. Their TEL architecture further incorporates caching and objectification layers to enhance response time.

However, while graph-based systems ensure correct dependency representation and scalability, they often lack intuitive visualization interfaces for exploring code structures, which creates usability challenges for developers working with unfamiliar projects.

## 2.3 Graph-Based Visual Programming

Graph-based visual programming represents software structures through interconnected graphical nodes rather than purely textual code representations. This paradigm enhances transparency by visually exposing relationships, dependencies, and execution flow within a codebase. While traditional code analysis research focuses on backend processing and structural extraction [1][2], visual analytics emphasizes human-centred interaction and usability. Visual interfaces reduce cognitive complexity and improve interpretability by presenting complex relationships in an intuitive format [15].

The principles of visual analytics suggest that graphical representations can significantly improve user understanding of complex software systems. However, the integration of graph-based visual interfaces directly into code analysis systems remains relatively underexplored in lightweight web-based platforms. The proposed system applies graph-based visualization techniques to code analysis, enabling users to explore project structures interactively while maintaining structured and accurate dependency representation.

## 2.4 No-Code and Low-Code Development Platforms

Low-code and no-code platforms aim to democratize software development by reducing reliance on manual programming. These platforms provide visual modelling tools, reusable components, and automated code generation mechanisms.

Recent studies analyse low-code and no-code approaches in application development, emphasizing abstraction, automation, and model-driven engineering techniques [5]. These approaches highlight improved accessibility and reduced development time as major benefits. However, they also identify limitations including reduced flexibility, restricted customization, and potential performance trade-offs. In contrast, traditional code analysis techniques primarily focus on structural correctness and accuracy but do not emphasize accessibility and usability. This reveals a gap between advanced code analysis methods and user-friendly developer tools. The proposed Coding Project Folder Analyzer seeks to bridge this gap by combining visual simplicity with structured and accurate analysis logic.

## 2.5 Static and Dynamic Code Analysis Techniques

With the growth of real-time analytics, ETL systems must increasingly handle continuous data streams rather than static batch datasets. Mehmood and Anees [4] conduct a systematic literature review on real-time big data stream processing, identifying key challenges such as low-latency processing, distributed coordination, scalability, and fault tolerance. Simitsis et al.

[2] discuss the transition from traditional ETL to more flexible ELT and streaming architectures, reflecting the evolving requirements of modern data ecosystems. While streaming frameworks address backend scalability, they often involve complex configuration and programming models. The visual construction of streaming pipelines remains an open usability challenge. Although the proposed system primarily focuses on batch processing, its DAG-based modular architecture aligns conceptually with principles used in streaming systems.

## 2.6 Data Visualization and Visual Analytics

Visualization is essential for transforming complex code structures into understandable insights. Visual analytics integrates computational analysis with interactive visualization to support human reasoning and interpretation. Research describes visual analytics as a multidisciplinary field combining data processing, visualization, and human-computer interaction techniques [3]. Interactive exploration enhances cognitive understanding and allows users to iteratively analyse and refine their interpretation of complex systems. Traditional code analysis approaches often separate structural analysis from visualization, limiting the ability of users to fully understand relationships within a codebase. Integrating visualization directly into code analysis systems can significantly improve transparency, debugging, and comprehension. The proposed system incorporates interactive graph-based visualization alongside analysis techniques, enabling users to explore dependencies and understand project structure more effectively.

# Chapter 3

# Limitations in Existing Systems

## 3.1 Limitations

### 3.1.1 Lack of Structural-Aware File Navigation

Standard file explorers—including VS Code and IntelliJ—represent project structures as linear directory trees. This approach fails to account for the logical "importance" of files, treating secondary CSS files or boilerplate configurations with the same visual weight as core entry points. When developers attempt to understand an unfamiliar codebase, they must manually distinguish between configuration, utility, and execution layers. No existing standard platform employs PageRank-based importance scoring to visually isolate critical business logic from peripheral code.

### 3.1.2 Absence of Automated Dependency Validation

None of the reviewed standard IDEs implement an automated, cross-file dependency validation pipeline during the exploration phase. While tools like "Find All References" exist, they require the user to proactively search for a specific term. They do not automatically detect circular dependencies or hidden coupling across different programming languages (e.g., a JavaScript frontend calling a specific Python backend route). The combination of AST-based parsing, multi-language regex detection, and graph-based conflict identification in our system represents

a novel approach not present in standard file-browsing systems.

### 3.1.3 Fragmented Code Analysis Without Relationship Mapping

Existing code search tools operate as isolated query engines. Identifying a logic change in one module does not automatically visualize the "ripple effect" or the Critical Path to other connected modules. The absence of system-scale relationship mapping represents a critical gap, given that software errors and architectural technical debt characteristically spread along dependency vectors. Our project's graph-integrated flow analysis addresses this gap by rendering import relationships as a dynamic digital landscape.

### 3.1.4 No Context-Accessible Architectural Interface

Current documentation tools universally assume the user has the technical literacy and time to navigate hundreds of pages of text-heavy files like READMEs or Javadoc's. Most code analysis solutions fail the "quick-start" demographic entirely. Platforms that do provide AI assistance typically offer line-by-line code snippets rather than a holistic, natural-language architectural summary that explains the project's purpose and framework composition to a new developer in under 60 seconds.

### 3.1.5 Missing Architectural Intelligence Layer for Management

No standard code exploration tool provides a dedicated "Project Manager" or "Architect" interface with repository-scale intelligence. Lead developers have no lightweight tool to monitor importance heatmaps, track complex dependency clusters, or visualize tech-stack distribution across a massive folder. This gap results in reactive rather than proactive architectural governance, with refactoring initiated only after significant technical debt has occurred, rather than in anticipation of emerging structural complexity.

### 3.1.6 Exclusion of Small Projects from Advanced Analytics

Advanced "Intelligence" tools for software (like enterprise static analysers) exist, but their participation requirements and high licensing costs create systemic barriers for student developers and small-scale researchers. Verification and maintenance costs of USD 1,000–5,000 per year make individual or classroom project participation economically impossible. Existing solutions do not provide a lightweight, web-based intermediary—pooling graph math and AI summarization in a simple, browser-based environment—necessary to make high-level insights viable for smallholders of code.

# Chapter 4

# Problem Statement & Objectives

## 4.1 Problem Statement

Modern software systems generate large and complex codebases from multiple sources such as enterprise applications, open-source repositories, web platforms, and cloud-based services. Before such codebases can be effectively used for development, debugging, or enhancement, they must undergo analysis steps including code parsing, dependency extraction, structural mapping, and functional interpretation. Code analysis systems are widely used to perform these operations and prepare software projects for efficient understanding and maintenance. Despite extensive research and development in code analysis technologies, several challenges still persist in existing systems. Many traditional tools are primarily designed for experienced developers and require advanced technical expertise for configuration and effective usage.

Analysing large codebases often involves manually navigating files, understanding dependencies, and interpreting logic using complex development tools, which creates a barrier for users who do not possess strong familiarity with the programming environment or project structure. Another limitation of conventional approaches is their lack of structured mechanisms for representing relationships between files and modules.

Although various techniques have been proposed to improve code analysis efficiency, these solutions typically focus on backend processing and algorithmic optimization rather than improving usability and accessibility for developers. Furthermore, modern software environments increasingly require scalable and flexible analysis systems. Configuring advanced analysis tools or integrating multiple frameworks often requires substantial programming expertise and complex setup procedures. At the same time, low-code and no-code platforms attempt to simplify interaction through graphical interfaces, but they may oversimplify analysis logic and limit flexibility for handling complex code structures. These challenges highlight a significant gap between technically advanced code analysis techniques and user-friendly developer tools. Therefore, there is a need for a system that combines structured analysis with an intuitive visual interface, enabling users to explore and understand codebases efficiently without requiring extensive programming expertise.

## 4.2 Objectives

The main objective of this project is to develop a Coding Project Folder Analyzer that simplifies the process of understanding and analysing software codebases through an intuitive graphical interface. The system aims to combine structured code analysis techniques with a user-friendly visual representation approach. The key objectives of the project are as follows:

- To design a visual graph-based interface that allows users to explore and analyse code structures without requiring extensive prior knowledge of the codebase.
- To implement a Directed Acyclic Graph (DAG)-based model that ensures accurate dependency mapping and clear representation of relationships between files and modules.
- To support multiple programming environments and file structures for flexible project analysis.
- To provide configurable analysis features that allow users to perform operations such as dependency detection, file importance ranking, execution flow tracing, and structural mapping.
- To enable analysis persistence and reusability by storing results in structured formats that can be saved, shared, and revisited.

# Chapter 5

# Proposed System & Methodology

## 5.1 System Overview

The proposed system is a web-based Coding Project Folder Analyzer designed to enable users to analyse, visualize, and understand software project structures efficiently. The overall operational flow of the system follows a user-driven analysis lifecycle model. Fig. 1 illustrates the flow of the system, describing how the system operates and the tasks it can perform.

The workflow begins with project upload or selection. Users are provided with two primary options: either they can upload a new project folder for analysis or open an existing analysis that has been previously saved or shared with them. Each project is represented as a structured graph of interconnected files and modules. These elements define operations such as dependency extraction, structural mapping, feature detection, and code relationship analysis. The connections between nodes determine the relationships and flow of dependencies within the project. Once analysis is completed or modified, it can be stored in a structured format. The stored representation includes file information, dependency mappings, structural relationships, execution flow, and other relevant metadata.

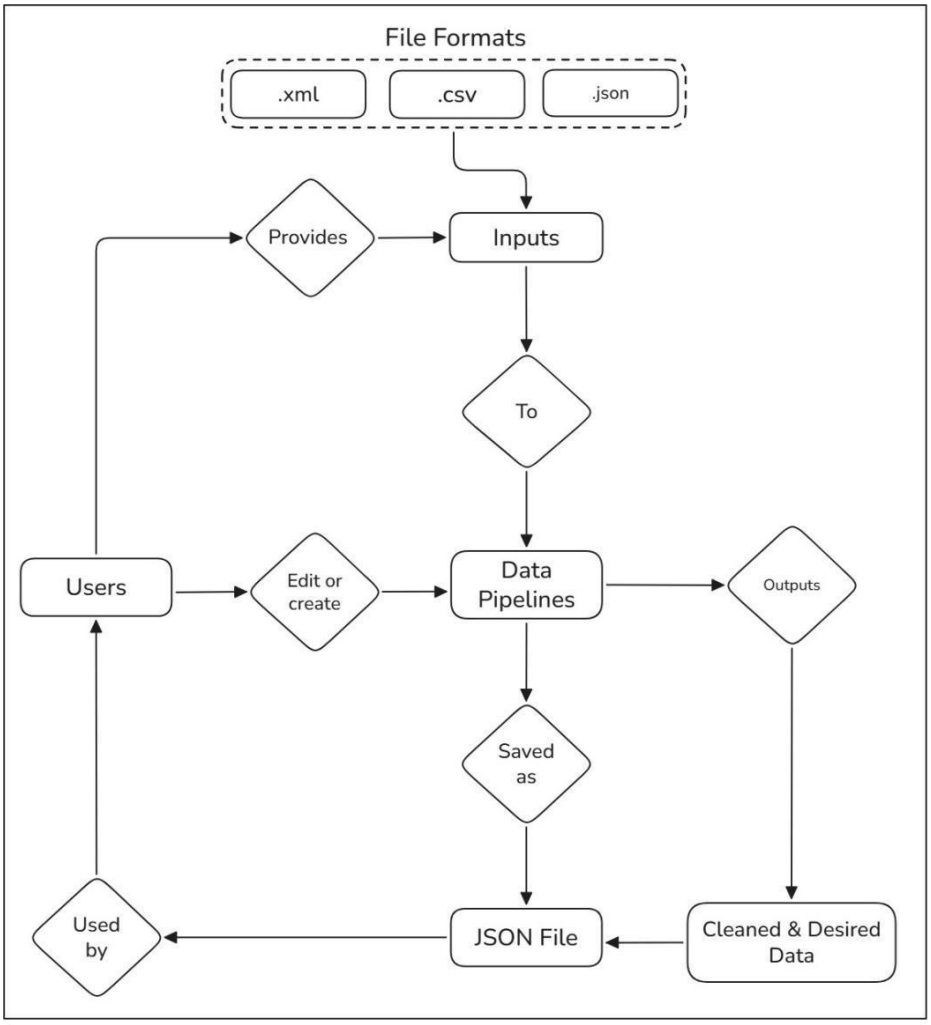


Figure 1: Overview of the system flow

This design enables portability, reusability, and sharing of analysis results. Saved analysis outputs can be reloaded and reused when required, supporting collaborative and iterative understanding of code structures. After a project is uploaded or loaded, the user provides input to the system in the form of a project folder or repository. The system supports multiple programming environments and file types, including Python, JavaScript, Java, and other commonly used source code formats.

Upon receiving the project input, the system parses the files, validates their structure, and extracts metadata required for code analysis. The project is then processed through the defined analysis workflow. The analysis engine processes the code according to the configured steps. Each module performs its respective operation, and intermediate results are passed to subsequent stages based on defined dependencies. After successful execution, the system generates the final analysis results.

This flexible input-output design ensures compatibility with diverse software environments and external systems. The overall system flow can therefore be summarized as: User → Upload/Select Project → Analyse Code → Generate Dependency Graph → Visualize Results

→ Save Analysis → Export/Share Results. By integrating structured code analysis, reusable result storage, multi-language input handling, and interactive visualization, the proposed system provides a flexible yet structured approach to software project analysis.

## 5.2 System Architecture

The System Architecture Layer is responsible for representing the project structure as a structured graph object. Each project is modelled as a directed graph, where:

Fig. 2 describes the overall structure of the system; it illustrates the hierarchy of all the components, the parameters associated with each component, and provides an understanding of their functionality within the code analysis process.

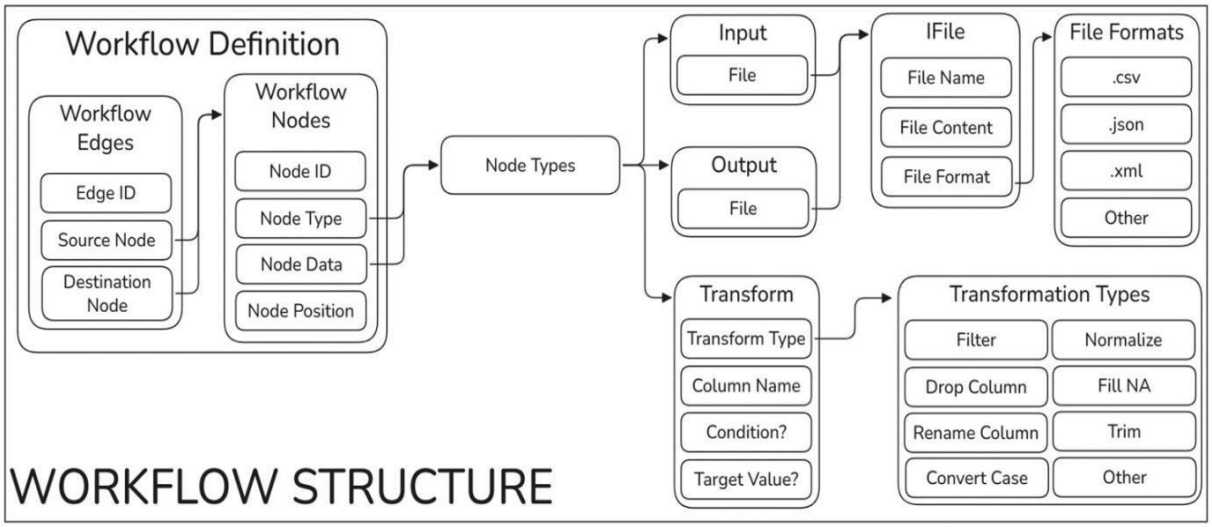


Figure 2: Overview of the System Architecture

### 5.2.1 Workflow Definition Layer

Each node in the system is modelled as a structured entity containing multiple attributes that define its behaviour and role within the code analysis process. Every node is assigned a unique identifier (Node ID) that distinguishes it from other nodes in the project graph. The Node Type specifies the functional category of the node, such as Input, Analysis, or Output, which determines the operational logic applied during execution. The Node Data attribute stores configuration parameters required for the analysis or processing task associated with the node. Additionally, Node Position information is maintained to preserve the visual layout of the node

within the user interface. Together, these attributes enable nodes to function as modular processing units that encapsulate both analysis logic and presentation metadata. The node type ultimately governs how the system interprets and processes each node during the code analysis workflow.

Graph edges represent the directional dependencies between nodes and are fundamental to defining the relationship structure of the project. Each edge is uniquely identified using an Edge ID, which allows precise tracking and management of node relationships within the graph structure. The Source Node attribute indicates the origin of the dependency, while the Destination Node attribute specifies the node that depends on the output of the source. By explicitly modelling these relationships, the system ensures that code dependencies are represented in a structured and consistent manner. The directed nature of edges enforces correct dependency mapping and prevents ambiguous relationships. Furthermore, this structured edge modelling enables the implementation of cycle detection mechanisms, ensuring that the graph remains acyclic and free from circular dependencies. As a result, the graph-based representation guarantees both structural correctness and reliable analysis behaviour.

### 5.2.2 Node Type Abstraction

The system architecture classifies analysis nodes into three primary categories: Input Nodes, Analysis Nodes, and Output Nodes. Each node type encapsulates specific responsibilities that contribute to the overall execution of the code analysis process. This categorization ensures clear separation of concerns and modularity within the system structure. By assigning distinct operational roles to each node type, the system maintains organized execution semantics and simplifies extensibility. The structured classification also enables validation mechanisms to verify correct analysis configuration before execution. As a result, the architecture promotes both clarity in system design and reliability during runtime processing.

The Input Node and Output Node serve as the entry and exit points of the analysis system, respectively. The Input Node is responsible for ingesting project data into the system and encapsulates a project abstraction that includes attributes such as Project Name, File Content, and File Type. The system supports multiple programming languages and file formats, ensuring compatibility with diverse software projects. Upon ingestion, the input module parses and standardizes the project structure into an internal representation to ensure consistent processing in downstream analysis stages. This unified representation simplifies analysis logic and reduces format-specific complexity. The Output Node, on the other hand, generates the final analysis

results after all processing steps have been completed. It encapsulates result generation functionality and supports exporting outputs in structured formats, thereby maintaining interoperability with external tools and systems.

The Analysis Node performs the core operations within the system. Each analysis node contains configurable parameters, including Analysis Type, Target File or Module, Condition, and Output Criteria, which collectively define the processing logic to be applied. These parameters allow dynamic configuration of analysis tasks through the visual interface. The system supports a variety of analysis operations, including code parsing, dependency extraction, file importance ranking, execution flow mapping, and structural visualization. Each operation works on structured metadata extracted during the input stage, ensuring parameter validation and consistency. By encapsulating analysis logic within modular nodes, the system enables flexible system configuration while maintaining deterministic and reliable execution behaviour.

## 5.3 Workflow Modelling Methodology

The proposed Coding Project Folder Analyzer models each project structure as a structured directed graph. The modelling methodology ensures deterministic analysis, structural validation, and extensibility of analysis operations.

### 5.3.1 Graph Based Workflow Representation

Each workflow is formally represented as a Directed Acyclic Graph (DAG):

$$G = (V, E)$$

Where, $V = \{v_1, v_2, v_3, \cdots, v_n\}$ represents the set of workflow nodes, $E \subseteq V \times V$ represents directed edges defining dependencies. Each edge $e = (v_i, v_j)$ indicates that node $v_j$ depends on the output of node $v_i$

The acyclic constraint ensures that

$$\forall v_i \in V, no\ path\ exists\ such\ that\ v_i \rightarrow \cdots \rightarrow v_i$$

This guarantees the absence of circular dependencies within the project structure. It ensures valid analysis ordering and prevents infinite dependency loops.

Before executing the analysis process, the system performs a structured validation and preparation phase to ensure correct and deterministic behaviour. Initially, adjacency mappings are constructed from the defined node and edge relationships, forming an internal graph

representation of the project. This mapping enables efficient traversal of dependencies and establishes the directional flow between connected components. Following this, the system computes the in-degree for each node, which represents the number of incoming dependenciesassociated with that node. The in-degree calculation is essential for identifying nodes that can be processed immediately without violating dependency constraints. Once adjacency relationships and in-degree values are established, the system applies a topological sorting algorithm to determine a valid processing order. This ordering guarantees that each node is analysed only after all its prerequisite nodes have been processed. As a result, the analysis remains cycle-free, structured, and consistent with the defined dependency graph.

### 5.3.2 Node Abstraction Model

Each node in the system is defined as a structured entity containing multiple attributes that determine its role within the code analysis process. Every node is assigned a unique identifier that distinguishes it from other nodes in the project graph. The node type specifies the functional category of the node, such as Input, Analysis, or Output, which determines how it behaves during execution. The node data stores configuration parameters required for performing analysis operations, while position information is maintained to preserve the visual layout of nodes within the user interface. This abstraction separates frontend visualization from backend processing, enabling a modular and flexible system design.

### 5.3.3 Analysis Modelling

Analysis nodes encapsulate configurable operations that process code structures step by step. Each analysis operation takes input from a previous stage and produces a new processed result that is passed to subsequent stages. Supported analysis operations include dependency extraction, code parsing, file importance ranking, execution flow mapping, and structural visualization. Each step generates an intermediate result while preserving previous states, ensuring consistency and traceability throughout the analysis process. This approach allows the system to maintain a structured workflow while enabling flexible and extensible analysis operations.

### 5.3.4 Code Preprocessing and Analysis Strategy

The preprocessing methodology implemented in the proposed system includes a structured approach for handling inconsistencies and variations within source code files. For each file and module in the project, the system applies standardized parsing and normalization techniques to preserve structural consistency and ensure accurate analysis. Following initial preprocessing,

additional analysis operations may be performed to enhance the quality and usability of the extracted information. These operations include removal of redundant or duplicate references, normalization of code structure for uniform representation, identification of key components, and extraction of meaningful relationships between modules. The system also applies techniques such as dependency mapping and structural grouping to improve clarity. Collectively, this structured analysis pipeline ensures consistent interpretation of code and prepares the project for accurate visualization and further analysis.

### 5.3.5 Column-Level Analysis Modelling

Code-level analysis operations within the proposed system are dynamically configured based on parameters received from the frontend interface. These operations enable fine-grained analysis of individual files and components within the project and allow users to tailor analysis logic to specific requirements. Supported operations include parsing code structures, identifying dependencies, renaming or grouping modules for clarity, detecting patterns, extracting function and class information, and removing unnecessary or redundant elements from the analysis view. Each operation is applied in a controlled and parameter-driven manner to maintain consistency within the analysis process.

Before applying any code-level analysis operation, the system performs structured validation checks to ensure correctness. These validations include verifying the existence of the specified file or module within the project, confirming compatibility of the analysis operation with the code structure, and checking parameter consistency to prevent invalid configurations. By enforcing these validation rules prior to execution, the system minimizes the risk of errors and inconsistencies during analysis. This validation-driven approach enhances robustness and ensures that analysis operations are applied safely and deterministically within the system.

### 5.3.6 Execution Ordering Methodology

The execution order of analysis nodes is determined through a structured traversal approach applied to the directed graph representation of the project. This approach ensures that nodes are processed in a dependency-respecting sequence, preserving the logical flow of relationships between interconnected files and modules. The process begins by identifying all nodes that have no incoming dependencies, meaning nodes that do not rely on any other components. These

nodes are eligible for immediate processing and form the starting points of the analysis. After processing each node, the system updates the dependency status of its connected nodes by reducing their dependency counts accordingly.

As dependencies are resolved, newly eligible nodes with no remaining dependencies are processed sequentially until all nodes in the graph have been analysed. This procedure guarantees that every node is processed exactly once and only after all prerequisite dependencies have been handled. Additionally, the system incorporates cycle detection mechanisms during graph validation. If a circular dependency is detected, the analysis is halted before execution begins. This ensures structural correctness and prevents undefined or inconsistent analysis behaviour within the system.

### 5.3.7 Analysis Serialization Model

Analysis results are serialized as structured data objects. This serialization model enables long-term persistence by allowing analysis outputs to be stored in a structured and portable format. By representing the analysis as a structured object, the system ensures that all node information, dependency relationships, and associated metadata are preserved accurately. This structured representation supports seamless sharing of analysis results between users, facilitating collaborative understanding and reuse. Additionally, serialized analysis data can be reloaded at a later stage for further exploration or re-analysis without reconstructing the process manually. The storage mechanism also supports version control, enabling iterative refinement while maintaining traceability of structural changes. By preserving node attributes, dependency mappings, and analysis configurations, the serialization model ensures reproducibility, portability, and consistency of results across different sessions and environments.

### 5.3.8 Methodological Characteristics

The proposed analysis modelling methodology ensures deterministic processing by enforcing a Directed Acyclic Graph (DAG) structure for representing project dependencies. By restricting the project structure to an acyclic graph, the system guarantees that each file or module is analysed only after its dependencies have been resolved, thereby preventing circular references and undefined analysis behaviour. This structural enforcement promotes predictable processing behaviour and maintains logical consistency within the system. Additionally, the methodology

incorporates modular node abstraction, where each node encapsulates specific functionality and configuration parameters. This modular design enhances clarity in system organization and simplifies maintenance. The separation of responsibilities between node types further strengthens reliability and structural organization.

Furthermore, the modelling framework supports analysis extensibility by allowing new analysis operations to be incorporated without altering the underlying system structure. The system also ensures format-independent handling by converting all supported code formats into a unified internal representation before processing. Prior to execution, validation mechanisms are applied to verify node connectivity, dependency integrity, and parameter consistency, thereby minimizing errors during analysis. Another key aspect of the methodology is the use of reusable serialized analysis definitions, which enable results to be stored, shared, reloaded, and version-controlled efficiently. Together, these characteristics provide a robust and scalable approach that balances flexibility, correctness, and reusability within the code analysis system.

# Chapter 6

# Experimental Setup

## 6.1 Frontend Setup

The frontend of the proposed system is developed using React along with TypeScript, providing a structured and type-safe environment for building the user interface. The application is built and served using Vite, which enables fast development and efficient build optimization.

The user interface is designed to support a graph-based analysis environment, where users can upload, analyse, and visualize coding project structures through an interactive interface. The frontend handles user interactions, project visualization, and communication with backend services.

### 6.1.1 Frontend Technologies Used

- **React** – Component-based UI development
- **TypeScript** – Type safety and improved code reliability
- **Vite** – Fast build tool and development server for efficient frontend development
- **React Router** – Client-side routing
- **React Icons** – Provides a collection of icons for enhancing the user interface
- **UUID** – Used for generating unique identifiers for nodes and components in the system

### 6.1.2 Frontend Structure

The frontend follows a modular structure consisting of:

- UI components for interface rendering
- State management for handling application data and interactions
- Visualization modules for graph-based project representation
- Service modules for API communication
- Utility modules for data handling

This setup ensures efficient interaction between the user interface and backend system while maintaining a scalable and maintainable frontend architecture.

## 6.2 Backend Setup

The backend of the proposed system is developed using Python, which provides a flexible and efficient environment for implementing code analysis and graph-based processing. The backend is responsible for handling project input, performing static code analysis, extracting dependencies, and communicating results to the frontend. The system implements a structured processing mechanism where project analysis is carried out based on defined dependency relationships. The backend manages analysis execution, applies algorithms, and ensures correct sequencing of operations.

### 6.2.1 Backend Technologies Used

- Python – Core programming language for backend processing
- Uvicorn – ASGI web server for handling requests
- NetworkX – Graph creation, dependency analysis, and PageRank implementation

### 6.2.2 Backend Structure

The backend is organized into multiple modules to separate concerns and improve maintainability:

- **API Layer:** Handles HTTP requests and responses through defined endpoints such as project upload, code analysis, and result retrieval.
- **Analysis Execution Engine:** Manages the execution of the analysis process by processing files and modules based on dependency relationships.

- **Code Analysis Module:** Performs operations such as AST parsing, dependency extraction, execution flow mapping, and file importance ranking using graph-based techniques.
- **Analysis Service Module:** Coordinates validation, execution flow, and interaction between different backend components to ensure structured and efficient analysis.

### 6.2.3 API Communication

The backend exposes RESTful API endpoints for:

- Project upload and analysis execution
- Individual analysis operations such as dependency extraction and parsing
- Code preprocessing and structural analysis
- Graph generation and visualization data

These APIs allow seamless communication between the frontend and backend, enabling users to upload projects, analyse them, and visualize results efficiently.

## 6.3 Development Environment

The development of the proposed Coding Project Folder Analyzer was carried out using a combination of modern software tools and frameworks to ensure efficient implementation, testing, and deployment. The environment was configured to support both frontend and backend development seamlessly.

### 6.3.1 Development Tools and Environment

- **Operating System:** Windows / Linux (depending on development setup)
- **Code Editor:** Visual Studio Code (VS Code)
- **Version Control:** Git and GitHub for source code management
- **Web Browser:** Google Chrome (for testing and debugging)

### 6.3.2 Software Requirements

- **Node.js** – Required for frontend development and package management
- **npm** – Dependency management for frontend modules
- **Python** – Required for backend development
- **pip** – Python package manager for installing backend dependencies

### 6.3.3 Development Setup

The frontend is executed using the Vite development server, typically running on http://localhost:5173. The backend server runs using Uvicorn, which handles API requests and manages communication between frontend and backend components. A virtual environment is used in Python to manage backend dependencies and ensure isolation. This development environment ensures smooth integration between the frontend and backend components, enabling efficient testing, debugging, and execution of the code analysis system.

# Chapter 7

# Results & Discussion

## 7.1 Results

### 7.1.1 Project Analysis Dashboard and Overview

The primary analysis interface of the Folder Analyzer was successfully implemented to provide a comprehensive post-processing overview. As seen in Figure 3, upon a successful project upload and analysis, the user is presented with a centralized dashboard that displays high-level metadata and technical snapshots of the codebase. This interface, built using a responsive React grid, integrates multiple specialized modules to offer a complete "health" snapshot of the repository. The Project Summarizer and File Handler components work in tandem to automatically identify the project type and specific frameworks, such as React or FastAPI, while calculating physical scale metrics like total file count and lines of code. This view further incorporates a localized breakdown of detected programming languages and an intelligent summary generated by the Gemini-Pro engine, which provides a natural-language explanation of the project's purpose and key architectural features. By centralizing these insights, the platform allows developers and administrators to understand the digital landscape of an unfamiliar codebase in a single view, significantly reducing the cognitive overhead required for technical debt assessment and project onboarding.

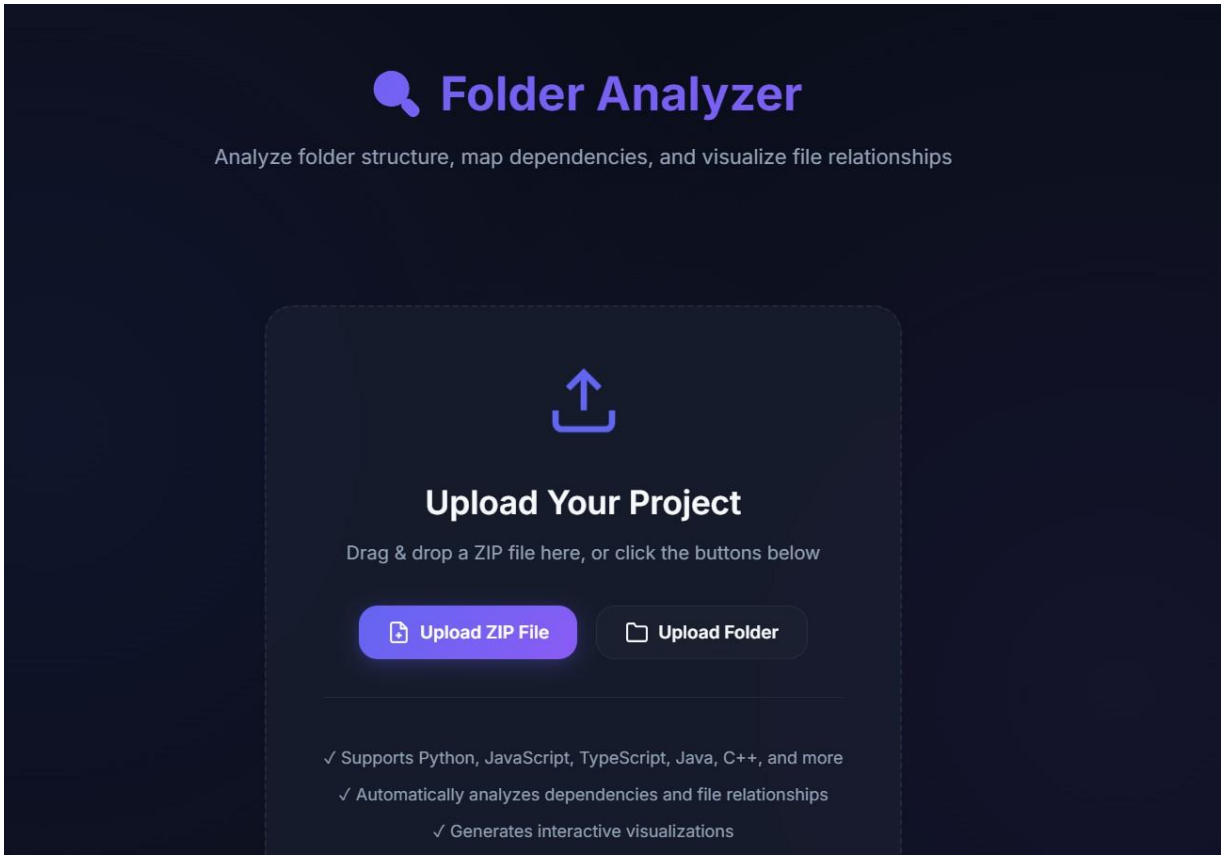


Figure 3 : Project Analysis Dashboard & Overview

### 7.1.2 AI-Powered Intelligent Summary

The intelligent summarization module was successfully implemented to bridge the gap between raw technical data and human-readable insights. As seen in Figure 4, the system utilizes an AI Summarizer module that integrates with the Google Gemini-Pro engine to generate comprehensive project overviews. In instances where the AI model is unavailable, the system automatically triggers a robust rule-based fallback mechanism, as indicated by the "Rule-Based" status badge in the interface. This panel provides a multi-layered analysis, including an executive summary of the project's scale—such as the identified file count and total lines of code—alongside a detailed breakdown of the technology stack and architectural organization. The interface specifically highlights "Key Components" identified by the Project Summarizer and presents a "Most Critical Files" list ranked by the Importance Scorer module, allowing developers to immediately identify high-priority files like models.py or utils.py. Furthermore, the panel concludes with actionable recommendations for project improvement, ensuring that developers receive clear guidance on best practices and potential optimizations for their specific codebase.

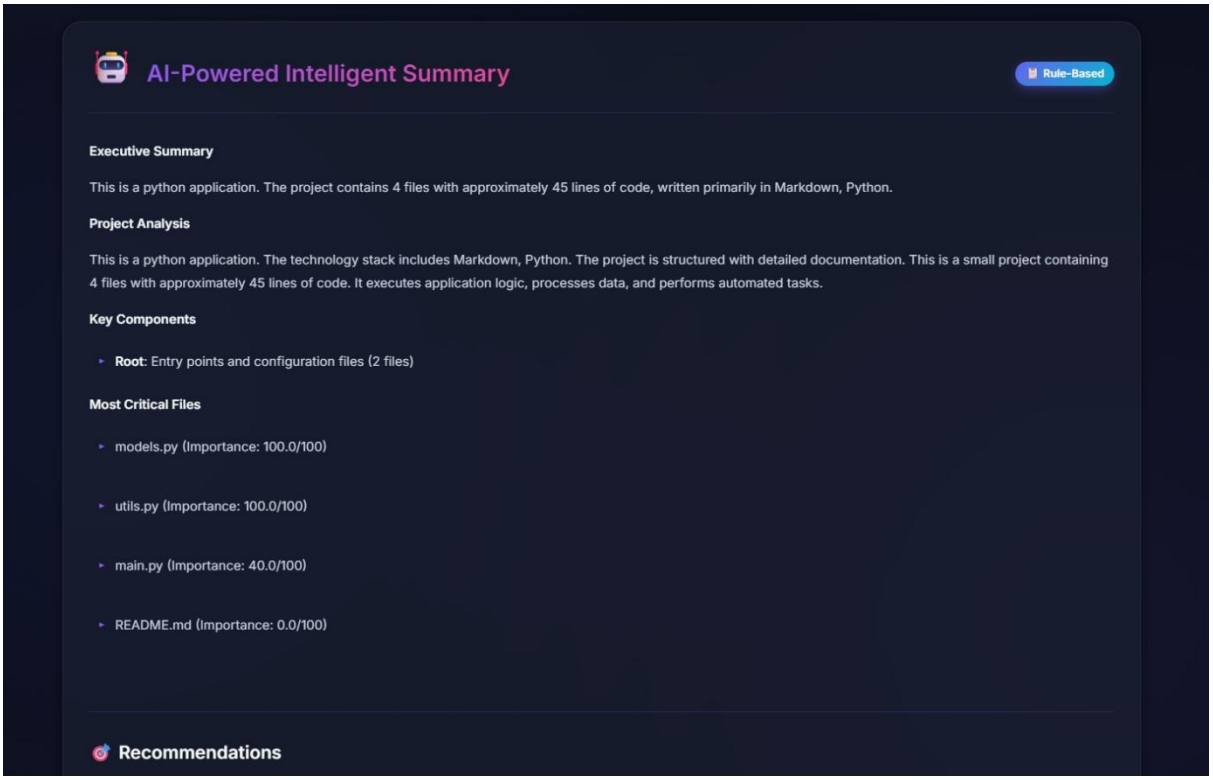


Figure 4 : AI-Powered Intelligent Summary

### 7.1.3 Dependency Mapping and File Relationships

The dependency mapping module provides a geospatial-like visualization of the project's internal architecture through an interactive node-link diagram. As seen in Figure 5, this interface renders a directed graph where nodes represent individual files and edges indicate import relationships, allowing developers to identify tightly coupled modules and architectural bottlenecks at a glance. This visualization is powered by the D3.js library on the frontend and the NetworkX graph engine on the backend, which processes the abstract syntax tree (AST) and regex-based import data extracted by the Dependency Analyzer. To improve scan ability, nodes are color-coded by their respective programming languages—such as Java (red) or Python/TypeScript (blue)—and their physical size is dynamically scaled based on the importance scores calculated by the PageRank-driven Importance Scorer. Users can interact with the graph through zooming, panning, and drag-and-drop repositioning, while hovering over specific nodes reveals direct incoming and outgoing connections to trace dependency chains. This visual representation transforms the complex text-heavy task of tracing imports into an intuitive discovery process, enabling proactive detection of circular dependencies and orphaned files within the digital landscape of the codebase.

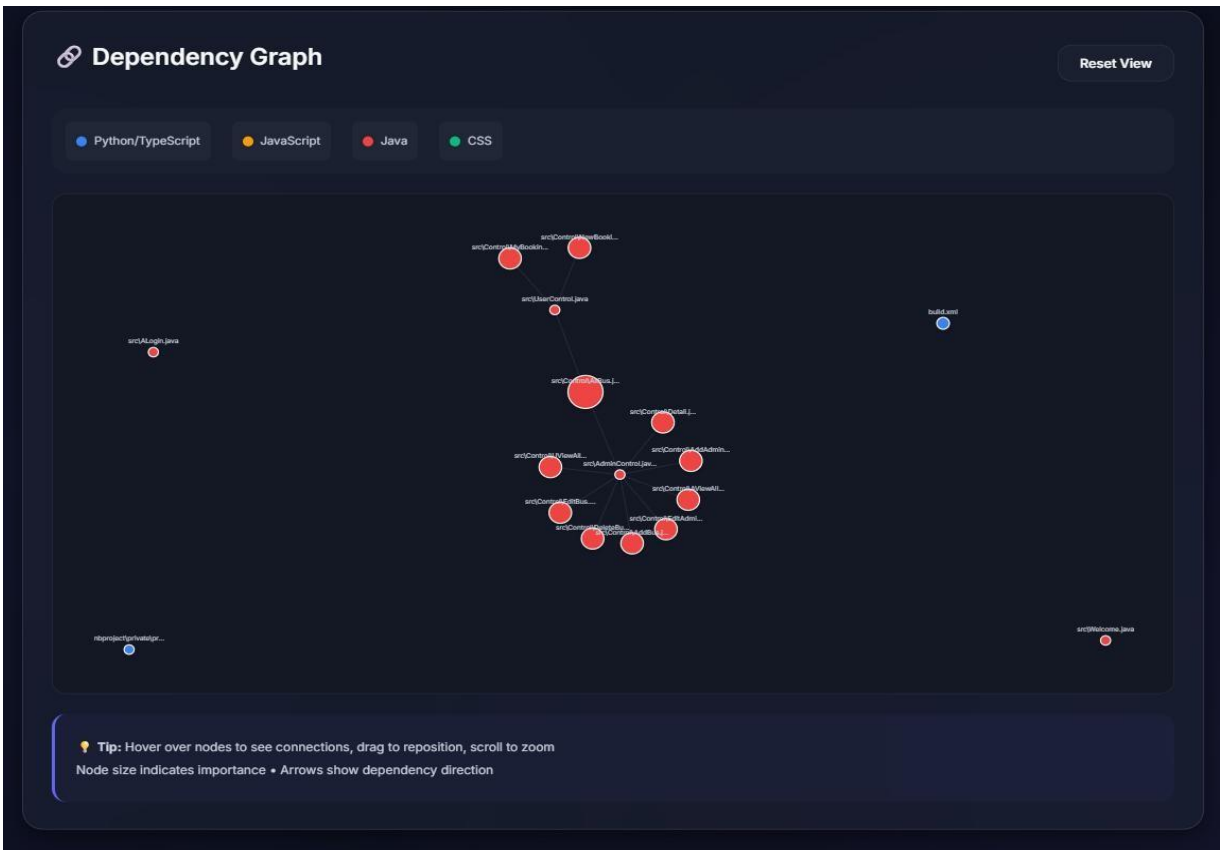


Figure 5 : Dependency Graph

### 7.1.4 Automated Project Categorization and Module Analysis

The project summarization module was successfully implemented to categorize and describe the uploaded codebase automatically. As seen in Figure 6, the "Project Summary" panel provides an immediate high-level overview of the application's nature and scale. This feature is driven by the Project Summarizer class, which analyses the repository's metadata and file distribution to detect the project type—in this instance, correctly identifying a "Java Application"—and its primary framework. The summary text is dynamically generated based on structural patterns, such as the presence of a src directory or specific configuration files. Key metrics, including the total number of files (22) and the total lines of code (5,042), are retrieved via the File Handler and presented in organized data tiles to quantify the project's scale. Furthermore, the "Key Modules" section utilizes directory-level analysis to infer the roles of specific folders; for example, the src directory is identified as containing "Source code," while the nbproject folder is categorized as a specialized module. This automated categorization significantly aids developers in understanding the functional boundaries of a new codebase without manual inspection.

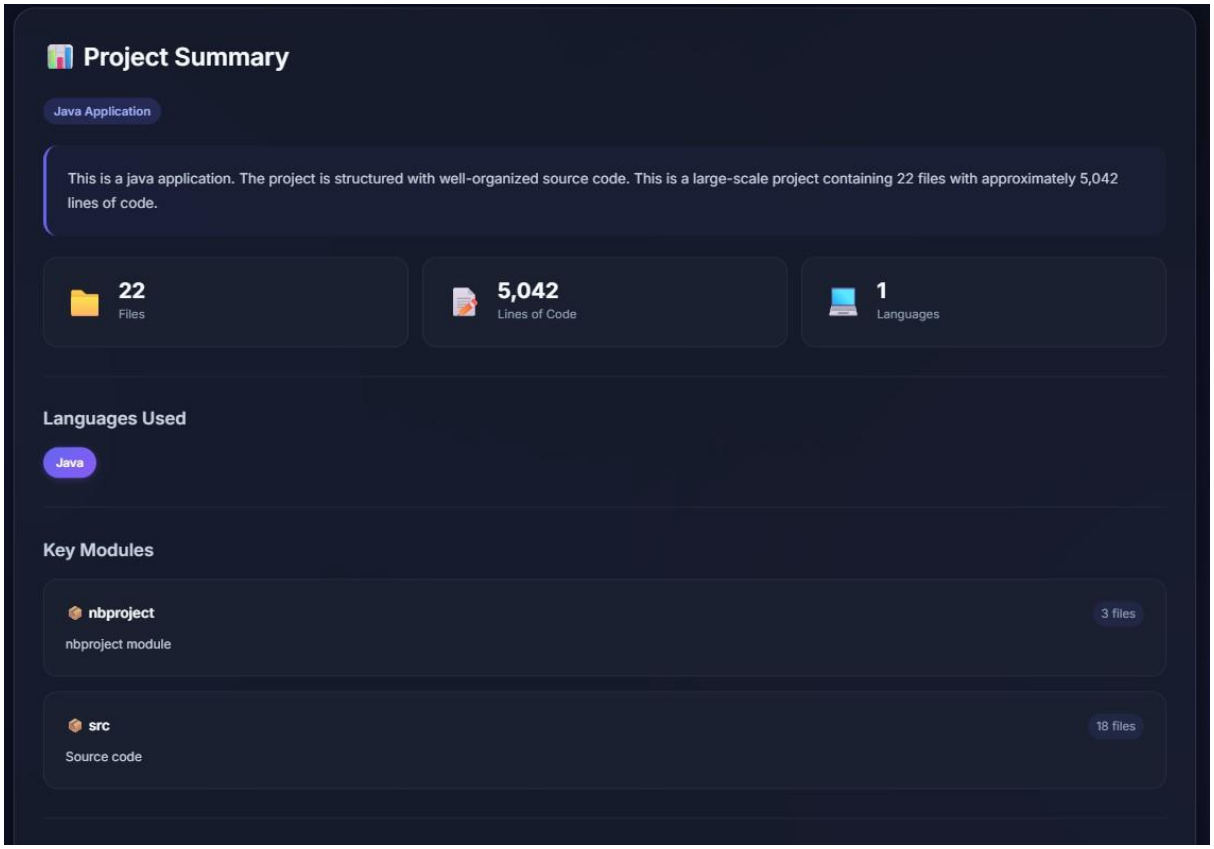


Figure 6 : Project Summary & Key Modules

## 7.2 Discussion

The results obtained from the implementation and testing of the Folder Analyzer confirm that the platform successfully achieves its core objective of delivering an integrated, AI-powered codebase management and visualization system. Each module—from file ingestion to deep architectural analysis—functioned as intended across various testing scenarios, including multi-language project uploads. The end-to-end workflow, beginning with the secure extraction of project files and concluding with the rendering of interactive dependency graphs and AI-generated insights, operated without critical failures, validating the robustness of the FastAPI and React-based architecture.

The interactive Dependency Graph proved to be an effective design decision, as utilizing a force-directed D3.js visualization transforms abstract import statements into a tangible digital landscape. This directly addresses one of the most persistent challenges in software development—understanding hidden coupling in unfamiliar codebases—by allowing developers to visually trace relationships and identify architectural bottlenecks in seconds. The integration of the PageRank-driven Importance Scorer further enhances this by dynamically

scaling node sizes, ensuring that critical utility files and entry points are immediately distinguishable from peripheral modules.

The AI-Powered Intelligent Summary module, supported by the Gemini-Pro engine, addresses a genuine documentation gap for developers onboarding onto legacy or complex projects. By providing a high-level executive summary and actionable recommendations alongside a rule-based fallback system, the platform ensures that users receive meaningful architectural context regardless of the availability of external AI APIs. This automated summarization, combined with the Project Summarizer's ability to detect frameworks and tech stacks, represents a significant advancement over standard IDE folder views that offer no inherent intelligence regarding a project's purpose.

Furthermore, the implementation of the Flow Analyzer with Mermaid.js integration provides a clear execution roadmap from identified entry points, which is critical for real-world debugging and flow verification. The ability to visualize the "Critical Path" and dependency chains in a single dashboard significantly reduces the technical barrier for engineers auditing large datasets of files. Overall, the results validate the proposed hybrid analysis architecture and confirm that each component—dependency mapping, importance ranking, and AI-driven insights—contributes measurably to reducing the cognitive overhead required for modern software exploration.

## 7.3 Usability & System Behaviour

The usability evaluation of the Coding Projects Analyzer shows that the visual interface makes project understanding much easier compared to manually reading code files. By presenting file relationships and structure through interactive graphs, the system reduces the effort needed to explore large codebases. The visual representation of files, dependencies, and important components helps users understand how the project is organized in a more intuitive way. This approach lowers the difficulty of analysing complex projects while still maintaining accurate results. As a result, users can focus on understanding the logic and structure of the project instead of spending time searching through files, which improves overall efficiency.

Several important usability observations were noted during testing. First, the system does not require deep programming knowledge to understand the project structure, making it useful for

students and beginners. Second, automatic dependency detection reduces errors by correctlyidentifying relationships between files and modules. Third, the generated summaries help users quickly understand the purpose of the project without reading the entire code. Finally, support for analysing different types of projects improves flexibility and usability. Overall, these features show that the system provides both accurate analysis and an easy-to-use interface for understanding software projects.

## 7.4 Limitations

While the proposed system works reliably for analysing small to medium-sized projects, there are some limitations that affect its use in large or complex environments. The current implementation is mainly designed for projects that can be processed within available system memory. Because of this, analysing very large codebases may slow down the system due to its dependence on in-memory processing. Also, the system does not currently support parallel or distributed analysis, which limits its ability to handle very large projects efficiently.

Another limitation is that the system does not support real-time updates, as it is designed to analyse projects only after they are uploaded, rather than tracking continuous changes. In addition, the performance of the system depends on system hardware and the size of the project being analysed. Even with these limitations, they highlight areas for future improvement. Adding support for distributed processing, parallel analysis, and real-time project tracking could greatly improve performance and make the system more suitable for large-scale and industry-level use.

# Chapter 8

# Conclusion & Future Scope

This project presented the design and development of a Coding Projects Analyzer using a visual and intelligent approach. The system combines code analysis techniques with an easy-to-use interface, allowing users to upload and understand project structures without needing deep programming knowledge. By representing file relationships through graphs, the system ensures clear visualization, proper dependency tracking, and avoids incorrect or confusing connections between files.

The proposed solution overcomes the limitations of traditional methods, where understanding a new codebase requires manually reading many files and takes a lot of time. With a visual and automated approach, the system simplifies project analysis while still maintaining accuracy and structure. It supports different types of projects and uses algorithms like PageRank to identify important files. It also provides summaries that explain the project in simple terms, making it easier for users to quickly understand the purpose and functionality of the codebase.

Testing with different project folders showed that the system correctly analyses file structures, detects dependencies, and generates meaningful visualizations and summaries. The graph-based approach ensures that relationships between files are properly maintained, while the modular design allows flexibility in analysing different types of projects also improves usability by reducing the effort needed to explore code, making it helpful for students,

developers, and researchers working with unfamiliar codebases.

Despite its usefulness, the system still has some limitations. It is mainly designed for analysing small to medium-sized projects and depends on in-memory processing, which may reduce performance when handling very large codebases. Also, the system does not support distributed processing or real-time updates, which limits its use in large-scale or high-performance environments.

Future improvements can focus on making the system more powerful and scalable. Integration with distributed technologies can help analyse large projects more efficiently. Adding support for real-time tracking of code changes can make the system more useful for active development environments. Advanced techniques, such as better ranking methods and improved analysis algorithms, can be added to enhance accuracy. AI-based suggestions can also help users automatically understand project structure and identify important files. Deploying the system as a cloud-based platform can allow multiple users to access it easily and improve resource management. Further improvements in visualization and monitoring, such as live updates and detailed views, can make the system more transparent and easier to use.

Overall, the proposed Coding Projects Analyzer successfully reduces the difficulty of understanding complex codebases by providing a simple, visual, and intelligent solution. It acts as a lightweight and flexible tool that can be useful for students, developers, and researchers working with different types of software projects.

# References


[1] S. Brin and L. Page, "The Anatomy of a Large-Scale Hypertextual Web Search Engine," Agricultural Systems, vol. 30, no. 1-7, pp. 107–117, 1998.

[2] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, "Compilers: Principles, Techniques, and Tools," 2nd ed., Pearson Education, 2006.

[3] J. Ferrante, K. J. Ottenstein, and J. D. Warren, "The Program Dependence Graph and Its Use in Optimization," ACM Transactions on Programming Languages and Systems, vol. 9, no. 3, pp. 319–349, 1987.

[4] S. Diehl, "Software Visualization: Visualizing the Structure, Behaviour, and Evolution of Software," Springer Science & Business Media, 2007.

[5] M. Bostock, V. Ogievetsky, and J. Heer, "D3: Data-Driven Documents," IEEE Transactions on Visualization and Computer Graphics, vol. 17, no. 12, pp. 2301–2309, 2011.

[6] S. R. Chidamber and C. F. Kemerer, "A Metrics Suite for Object Oriented Design," IEEE Transactions on Software Engineering, vol. 20, no. 6, pp. 476–493, 1994.

[7] A. Vaswani, N. Shazeer, N. Parmar, et al., "Attention is All You Need," in Proc. Advances in Neural Information Processing Systems (NIPS), pp. 5998–6008, 2017.

[8] M. Chen, J. Tworek, H. Jun, et al., "Evaluating Large Language Models Trained on Code," arXiv:2107.03374, 2021.

[9] Google Gemini Team, "Gemini: A Family of Highly Capable Multimodal Models," Technical Report, Google DeepMind, 2024.

[10] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring Network Structure, Dynamics, and Function using NetworkX," in Proc. 7th Python in Science Conference (SciPy), pp. 11–15, 2008.

[11] S. Tiangolo, "FastAPI: High performance, easy to learn, fast to code, ready for production," fastapi.tiangolo.com, 2024.

[12] Meta Open Source, "React: A JavaScript Library for Building User Interfaces," react.dev, 2024.

[13] B. Johnson and B. Shneiderman, "Tree-Maps: A Space-Filling Approach to the Visualization of Hierarchical Information Structures," in Proc. IEEE Visualization, pp. 284–291, 1991.

[14] J. Maletic, A. Marcus, and M. L. Collard, "A Taxonomy of Software Visualization," in Proc. 14th IEEE International Conference on Software Engineering and Knowledge Engineering, pp. 593–600, 2002.

[15] W. Cunningham, "The WyCash Portfolio Management System," OOPSLA Experience Report, 1992.

[16] Z. Feng, D. Guo, D. Tang, et al., "CodeBERT: A Pre-Trained Model for Programming and Natural Languages," in Proc. Findings of the Association for Computational Linguistics: EMNLP, 2020.

[17] A. Marcus and T. Mende, "Visualizing Software Changes," in Proc. 25th IEEE International Conference on Software Engineering, 2003.

[18] Mermaid.js, "Mermaid: Generation of diagrams and flowcharts from text in a similar manner as markdown," mermaid.js.org, 2024.

[19] M.-A. Storey, F. D. Fracchia, and H. A. Müller, "Cognitive design elements to support the construction of a mental model during software exploration," Software Quality Journal, vol. 8, pp. 19–38, 1999.

[20] V. J. Nair, "Static Analysis Techniques for Modern Software Engineering," APSIT Technical Journal, vol. 4, no. 1, 2024.